ARCHITECTURE BLUEPRINT

DRAFT
NOT APPROVED

DZen Doctor App

System architecture and full feature scope, laid out for review — not a build plan. Every item here needs the doctor's sign-off before a single line of production code gets written.

PROJECT DZen Doctor App	CLIENT Solo Physiotherapy Practice	DRAWING NO. ARCH-001	REVISION A — Draft
DEVICES iPhone + Android Tablet	PLATFORM Flutter + Firebase	STATUS For Doctor Review	DATE 6 July 2026

00 · HOW TO READ THIS DOCUMENT

Legend / Key

Every screen in this document is explained twice: once in plain English with an example (for the doctor), and once in technical detail (for the dev). Watch for these three markers throughout:

- **Confirmed Requirement**
  Comes directly from what the doctor described. Not up for debate — just needs a final "yes, that's right."
- **Assumption / Technical Decision**
  Something the doctor described loosely, where a specific interpretation had to be picked. Needs a quick confirm.
- **Suggested Feature**
  An idea nobody asked for yet, added because most physio practices need it. Entirely optional — treat as a proposal, not a plan.

01 · SYSTEM OVERVIEW

The Big Picture

🗣 SPEAKING TO THE DOCTOR

Your app works like a shared notebook that lives on the internet instead of on paper. Your iPhone and your Android tablet are just two windows looking into the *same* notebook. Add a new visit on the tablet, and it'll show up on your phone within a second or two — and the other way around too.

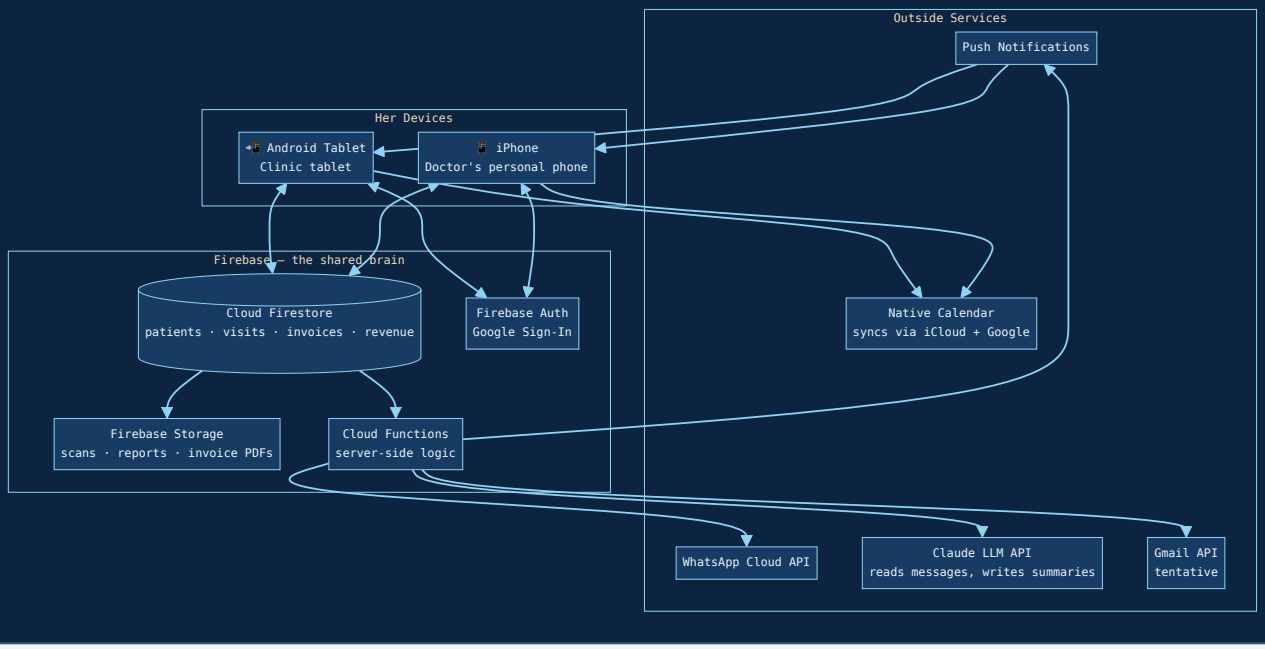
If there's no internet for a moment — say, you're inside a patient's building during a home visit — your app keeps working using a temporary local copy, and quietly catches up the moment the connection comes back. You won't lose anything.

🛠 SPEAKING TO YOU, THE DEVELOPER

You'll ship a single Flutter codebase compiled to iOS and Android binaries. Keep all app state in `Cloud Firestore` with native offline persistence enabled — writes queue locally and sync opportunistically. Wire your Riverpod providers to subscribe to Firestore snapshot streams, so both device instances re-render in real time off the same backend.

Scope data with Firebase Auth (Google Sign-In) to a single doctor UID. Key every collection by an `ownerId` field even though only one account exists today — costs you nothing now, and keeps the door open if a second practitioner or assistant account is ever needed.

FIG. 1 — SYSTEM ARCHITECTURE



Two devices, one brain

🗣 SPEAKING TO THE DOCTOR

Your iPhone and your Android tablet are just two doors into the same room. Nothing is stored "only on the tablet" or "only on the phone" — everything lives centrally, so either device always shows you the full, current picture.

🛠 SPEAKING TO YOU, THE DEVELOPER

No local SQLite or per-device database — Firestore is the single source of truth, with its built-in offline cache doing the heavy lifting for spotty connectivity, exactly like native offline-first apps behave.

02 · FEATURE BREAKDOWN

The Six Screens

Every screen below maps directly to what was described. Fields, flows, and flagged decisions are listed under each.

SCREEN 01 / 06

Client Records

A folder per patient — who they are, their condition, and their session history.

🗣 SPEAKING TO THE DOCTOR

The Client Records page will be like a digital file cabinet with one folder per patient. Each folder shows: name, age, gender, the condition being treated (e.g. "lower back pain"), home address, a phone number plus a backup contact number, every session they've had so far, what's coming up next, whether they have a pending payment, paying per session or as a bundle (e.g. paying for 3 sessions together instead of one at a time). It will be laid out neatly so you can go through it easily without getting overwhelmed.

🛠 SPEAKING TO YOU, THE DEVELOPER

A `patients` collection: name, age, gender, condition, address, phone1, phone2, paymentType enum(`perSession` , `package`), createdAt. Session history isn't embedded here — it's queried live from the `visits` collection where `patientId` matches, split into completed vs. upcoming. Keeps the visit record as the single source of truth instead of duplicating data in two places.

KEY FIELDS

Name | Age | Gender | Condition | Address | Phone (primary) | Phone (secondary) | Payment type | Session history | Upcoming sessions

- **Assumption:** "Package" is treated as more than a label — see Suggested Feature #1 (Package Balance Tracker) for whether you want the app to actually count down remaining sessions in a bundle.

SCREEN 02 / 06

Online Sessions

Her upcoming video appointments — Google Meet or Zoom.

🗣 SPEAKING TO THE DOCTOR

The Online Sessions screen is a separate list showing only your video call appointments. For example, when a patient can't travel in for a visit but needs guidance on exercises, or you want to look at an X-ray together over video before deciding next steps, this screen shows those Meet/Zoom appointments coming up, with the link right there so you can tap and join immediately.

🛠 SPEAKING TO YOU, THE DEVELOPER

Functionally the same `visits` collection as Screen 3, filtered where `mode == "online"` . Each online visit doc adds a `meetingLink` field and an optional `attachmentIds` array pointing to scan images in Firebase Storage you may want on hand during the call. The app does not host the video call itself — Meet/Zoom remain external.

- **Assumption:** Treating this as a filtered view of the same appointment data as Screen 3, rather than a separate structure — simpler to build and keeps revenue/reporting consistent. Confirm no online-only fields are missing (e.g. which platform, Meet vs. Zoom).

SCREEN 03 / 06

Upcoming Visits (Home Visits)

Her daily/weekly line-up of in-person visits, with maps and rescheduling built in.

🗣 SPEAKING TO THE DOCTOR

The Upcoming Visits screen shows your lineup of home visits — where you're going, when, for how long, and to see whom. Tapping a patient's address opens Maps and starts directions instantly, no copy-pasting needed. If you need to move a visit — say from Tuesday 3pm to Wednesday 5pm — you just edit it right there, and the patient *automatically* gets a WhatsApp message about the new time. You never have to type a text yourself.

🛠 SPEAKING TO YOU, THE DEVELOPER

Same `visits` collection, filtered `mode == "inPerson"`. Fields: date, time, estimatedDurationMinutes, patientId, address, status enum(`upcoming`, `completed`, `missed`, `cancelled`). Address tap opens a maps deep link via Flutter's `url_launcher` (`geo:` on Android, `maps:` on iOS). Editing date/time fires a Cloud Function that diffs old vs. new timestamp; on change, it sends the "reschedule" WhatsApp template and updates the linked calendar event.

KEY FIELDS

Date & day | Time & duration | Patient name | Address → Maps | Editable schedule | Auto WhatsApp on reschedule

SCREEN 04 / 06

Revenue

What she earned, what she spent, filtered by week or month.

🗣 SPEAKING TO THE DOCTOR

The Revenue screen is your money dashboard. It shows what you've earned recently, broken down by home visits, online sessions, and other sources, with a simple toggle for "this week" vs. "this month." There's also a separate section for your expenses, so income and spending are visible at a glance.

🛠 SPEAKING TO YOU, THE DEVELOPER

Reads from precomputed `stats/{yyyy_mm}` aggregate docs, updated via Cloud Function triggers on invoice writes inside a Firestore transaction — avoids summing potentially hundreds of invoices on-device every time the screen opens. A separate `expenses` collection holds manually entered outgoing costs (date, amount, description).

- **Assumption:** Expenses are manually typed in by you (e.g. "bought supplies, ₹2,000") — there's no bank account connection. Also assuming weekly totals are calculated on the fly from the current month's data rather than separately precomputed; flag if you need fast weekly views going back further than a month.

SCREEN 05 / 06

Dashboard

Her homepage — snapshot of money, schedule, and calendar, all in one place.

🗣 SPEAKING TO THE DOCTOR

The Dashboard is the first thing you'll see when you open the app: a quick revenue snapshot, recent visits, recent money in and out, upcoming appointments, and (once it's built) a short AI-written summary of WhatsApp chats with patients. It also shows your calendar with reminders — and here's the clever part: instead of building a brand-new calendar system, the app simply adds your appointments into the Calendar app already on your phone and tablet. Since your iPhone is signed into your Apple ID and

the tablet into a Google account, those apps already sync automatically — the app just piggybacks on something that already works.

🛠 SPEAKING TO YOU, THE DEVELOPER

Dashboard aggregates read-only summaries: `stats/{current period}` for revenue, a limited `visits` query for recent/upcoming, `expenses` for recent debits, and a stubbed `agentSummaries` collection — empty until the WhatsApp agent is built, so the card simply renders "No summary yet" or hides itself, meaning no UI rework later. Calendar sync uses the Flutter `device_calendar` plugin to write events straight into the native OS calendar on each device, rather than integrating Apple's CalDAV or the Google Calendar API server-side — no OAuth calendar scopes needed, and it "just works" as long as each device stays signed into its own account.

- **Assumption:** Chose device-native calendar sync (simple, no extra logins) over a full cloud Calendar API integration. This is the right call unless you want visits visible on a calendar on some other device/computer you don't carry — worth a direct confirm since it shapes how "the calendar" is built.

SCREEN 06 / 06

Invoice Creation

Bill one visit, or bundle several — sent with a single tap.

🗣 SPEAKING TO THE DOCTOR

The Invoice Creation screen lets you create a bill after one or several visits and send it straight to the patient with a single tap. You can bill one visit at a time, or bundle a few together — for example, if a patient wants to settle their last 5 visits at once, you select those 5 and hit "Generate Invoice." The invoice lists every date and what was charged for each, then totals everything — so there's no confusion about what's being paid for.

🛠 SPEAKING TO YOU, THE DEVELOPER

An `invoices` collection: patientId, visitIds (array — one or many), lineItems (derived: date + description + amount per visit), totalAmount, generatedAt, sentVia enum(`whatsapp`, `email`, `both`), status

enum(`draft` , `sent` , `paid`). PDF built with the `pdf` + `printing` Flutter packages, uploaded to Firebase Storage, then attached to a WhatsApp "invoice-ready" template message and/or emailed via Gmail API.

- **Assumption:** Once a visit is included in an invoice, it gets flagged "invoiced" so it can't accidentally be billed twice. Confirm this matches expectations — e.g. what should happen if you need to edit an invoice after sending it.

03 · SEEING IT IN ACTION

Real-World Examples

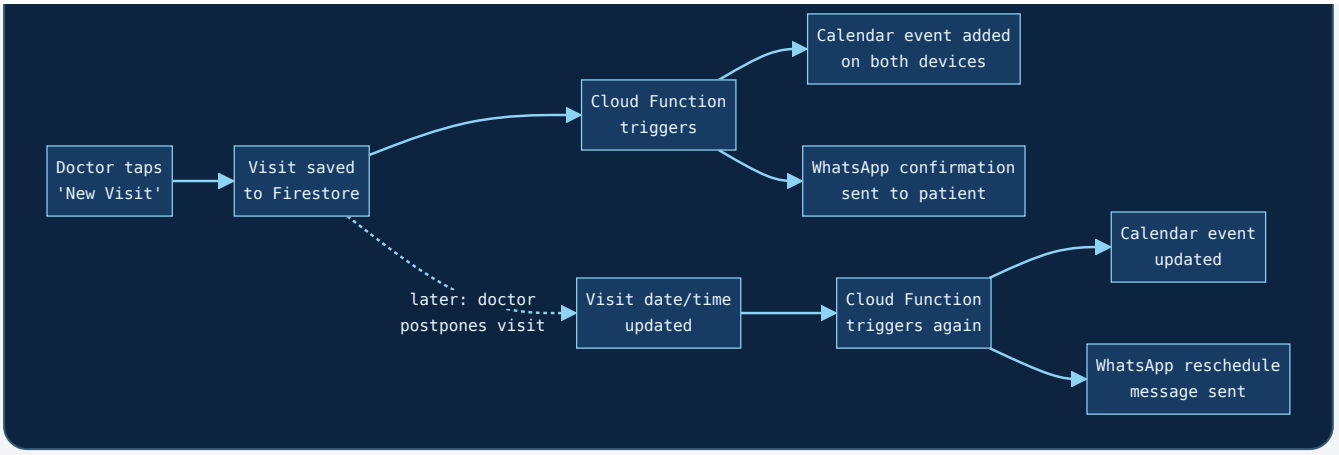
Three everyday scenarios, walked through step by step, so it's easier to picture how the pieces connect.

EXAMPLE A

Booking, then rescheduling, a home visit

1. **You book a visit** — On the tablet, you add a new visit for Mr. Sharma: Tuesday, 4:00 PM, 45 minutes, his home address.
2. **It appears everywhere** — Your iPhone shows the same visit within seconds. It also gets added to the Calendar app on both devices.
3. **Patient gets confirmation** — A WhatsApp message goes out automatically: "Your visit is confirmed for Tuesday 4:00 PM."
4. **Plans change** — You later move it to Wednesday, 5:00 PM, right from the Upcoming Visits screen.
5. **Patient is told automatically** — A "your visit has been rescheduled" WhatsApp message goes out, and both calendars update — you never have to type a message yourself.

FIG. 2 — BOOKING & RESCHEDULE FLOW

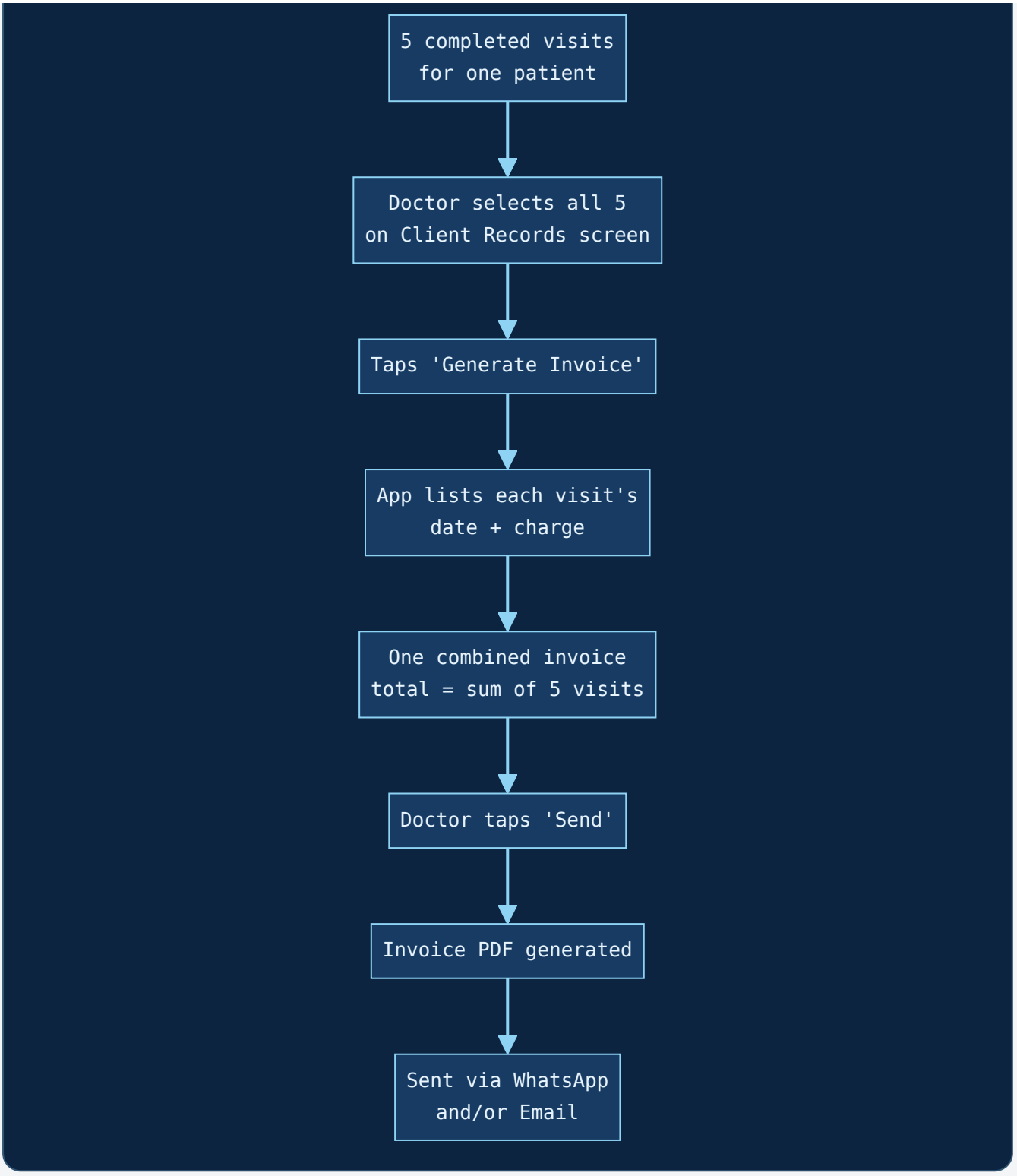


EXAMPLE B

Billing five visits together

1. **Five visits pile up** — Mr. Sharma has completed 5 sessions and now wants to pay for all of them at once.
2. **You select them** — On his Client Records page, you select the 5 completed, not-yet-billed visits.
3. **One invoice, all dates listed** — Tapping "Generate Invoice" builds a single bill listing each date and its charge, plus a grand total.
4. **Sent with one tap** — You tap "Send," and the patient receives the invoice on WhatsApp (and/or email) immediately.

FIG. 3 — CLUSTERED INVOICE FLOW



EXAMPLE C

A patient replies on WhatsApp (before the AI agent exists)

1. **Patient texts back** — After a reminder, a patient replies "Can we move it to 5pm instead?"
2. **Message is logged** — The reply is captured and stored in the patient's WhatsApp history.

3. **Claude reads it for context** — The LLM reads the message to produce a short summary line ("asked to move Tuesday visit to 5pm") — but does not reply or reschedule on its own yet.
4. **You see it on the Dashboard** — You see the summary and handle it yourself for now — either replying on WhatsApp directly or editing the visit in-app.
5. **Later:** once the separate WhatsApp AI agent is built, this same pipeline can be upgraded to let it draft or send replies automatically — no data model changes required.

04 · EXTERNAL CONNECTIONS

Behind the Scenes

The services the app talks to outside of itself, and how each one is used.

WhatsApp Messaging

🗣 SPEAKING TO THE DOCTOR

Your patients get automatic WhatsApp messages at the right moments: booking confirmed, a reminder before the visit, a cancellation, a reschedule, "your invoice is ready," and "payment received." You don't type any of these yourself — the app sends them the moment the matching action happens.

🛠 SPEAKING TO YOU, THE DEVELOPER

Meta WhatsApp Cloud API, called only from Cloud Functions using pre-approved templates: confirmation, reminder, cancellation, reschedule, invoice-ready, payment-received. Inbound replies land via a webhook → Cloud Function → Claude LLM parses intent + writes a summary line into the patient's timeline (see Example C above).

- **Assumption:** Reminder timing (e.g. 24 hours before a visit, or the morning of) hasn't been specified — needs a direct answer from you.

Calendar Sync

🗣 SPEAKING TO THE DOCTOR

When you book a visit in the app, it automatically gets added to the Calendar app that's already on your phone and tablet — the exact same one you'd use to check "what's on today." No separate calendar to learn or check.

🛠 SPEAKING TO YOU, THE DEVELOPER

Flutter `device_calendar` plugin writes events to the native calendar store on each OS. iOS syncs via her Apple ID/iCloud; Android syncs via the tablet's signed-in Google account. No calendar-specific OAuth needed on the app's side.

Email

🗣 SPEAKING TO THE DOCTOR

This part isn't fully decided yet. The current idea is to send a copy of the invoice by email too (in case you want a proper paper trail beyond WhatsApp), but we haven't confirmed if you need it.

🛠 SPEAKING TO YOU, THE DEVELOPER

Tentative: Gmail API triggered from Cloud Functions, same pattern as an existing integration. Alternatives on the table: SendGrid, or Firebase's Trigger Email extension. Nothing is locked in.

- **Assumption:** Needs a direct decision — is email needed at all if WhatsApp already covers invoices, or is it specifically wanted for your own records/accounting?

Push Notifications

🗣 SPEAKING TO THE DOCTOR

Both your phone and tablet will buzz with a notification the moment there's a new booking or a patient reply — so you don't have to keep the app open to know something's happened.

🛠 SPEAKING TO YOU, THE DEVELOPER

Firebase Cloud Messaging (FCM), triggered from the same Cloud Functions that handle bookings and inbound WhatsApp messages — sent to both registered device tokens.

Future Extensibility Points (not built now — architecture just leaves room)

🗣 SPEAKING TO THE DOCTOR

The WhatsApp AI agent and calling agent are being built separately and plugged in later, as agreed. If you have a website with a "Contact Me" button, a patient who reaches out through it will show up in the app just like a new WhatsApp message does. Neither feature is built today, but nothing in this plan blocks adding them later.

🛠 SPEAKING TO YOU, THE DEVELOPER

The Cloud Function layer already sits between WhatsApp and the app doing simple template sends + logging — that same layer can later be swapped to let an agent draft/send replies or trigger calls, with zero changes to the patients/visits/invoices data model. For the website, a public Cloud Function endpoint can write into a new `leads` collection; the Dashboard shows "New website inquiry from [Name]" the same way it shows a new WhatsApp message today.

05 · UNDER THE HOOD

Data Storage Map

For the technical side: every major collection the app stores data in, what it's for, and its key fields. The doctor doesn't need to read this table closely — it's here for reference.

COLLECTION	WHAT IT HOLDS	KEY FIELDS
patients	One record per patient	name, age, gender, condition, address, phone1, phone2, paymentType, packageBalance*

COLLECTION	WHAT IT HOLDS	KEY FIELDS
`visits`	Every visit — home, online, or clinic	patientId, mode, date, time, durationMins, address, meetingLink, status, invoiced (bool)
`invoices`	Every bill generated	patientId, visitIds[], lineItems[], totalAmount, sentVia, status
`stats`	Pre-added-up revenue totals per period	period, totalRevenue, breakdown by visit mode
`expenses`	Money going out	date, amount, description, category*
`whatsappLogs`	Every WhatsApp message, in and out	direction, patientId, text, timestamp, llmSummary
`agentSummaries`	Placeholder for future AI agent output	empty/stub until the WhatsApp agent is built
`leads*`	Future: website inquiries	name, contact, message, source

* Fields marked with an asterisk are tied to a Suggested Feature or a future integration — see Section 06.

06 · PROACTIVELY IDENTIFIED GAPS

Suggested Features for Validation

These weren't asked for. They're patterns that come up in almost every physio practice, added here so nothing important gets missed — each one needs an explicit yes, no, or "maybe later" from the doctor. None of these are assumed to be part of the plan.

SUGGESTED 01

Package Balance Tracker

Knows how many sessions are left in a paid bundle (e.g. "2 of 3 used") and gently flags when a client is on their last session.

Why it matters: without this, "package" is just a label with no way to track what's actually owed or remaining.

SUGGESTED 02

Session / Treatment Notes

A small notes box on each visit — what was done, what you observed. E.g. "worked on shoulder mobility, patient reports less pain."

Why it matters: helps track a patient's progress over weeks without relying on memory.

SUGGESTED 03

Missed / Cancelled Visit Tracking

A distinct status for a no-show or cancellation, separate from "completed" — so it doesn't wrongly count against a package or get billed.

Why it matters: keeps revenue and package counts accurate.

SUGGESTED 04

Payment Mode Tracking

Recording whether a payment came in as cash, UPI, card, or bank transfer.

Why it matters: useful for your own accounting/reconciliation later.

SUGGESTED 05

Recurring Visit Scheduling

Book a repeating slot — e.g. every Monday for 4 weeks — in one go, instead of adding each visit one at a time.

Why it matters: saves real time for regular, ongoing patients.

SUGGESTED 06

Simple Patient Search

Type a name to instantly find a patient, instead of scrolling the full list.

Why it matters: becomes essential once your patient list grows past a couple dozen names.

SUGGESTED 07

App Lock (PIN or Fingerprint)

An extra lock screen on the app itself, on top of the device's own lock.

Why it matters: patient health and payment info is sensitive, and the tablet may sit out in the clinic where others could pick it up.

SUGGESTED 08

Backup / Export to Excel

A button to download all patient and revenue data as a file, for your own offline records or an accountant.

Why it matters: cheap peace of mind, and useful outside the app entirely.

07 · SIGN-OFF

Master Validation Checklist

Use this as a reference when discussing with the doctor. No embedded feedback — just a list of what needs a decision.

Does each screen match what you described?

SCREEN	DECISION NEEDED
01 · Client Records	Yes / Needs changes
02 · Online Sessions	Yes / Needs changes
03 · Upcoming Visits	Yes / Needs changes
04 · Revenue	Yes / Needs changes
05 · Dashboard	Yes / Needs changes
06 · Invoice Creation	Yes / Needs changes

Assumptions needing a direct confirm

#	ASSUMPTION	TYPE
1	Package tracking needs an actual running balance, not just a label	assumption
2	Online Sessions is a filtered view of the same visit data as home visits	assumption
3	Expenses are manually entered; no bank integration	assumption
4	Calendar sync uses the phone/tablet's native calendar, not a direct cloud Calendar API	assumption
5	A visit gets flagged "invoiced" once billed, to avoid double-billing	assumption
6	WhatsApp reminder timing (how far before a visit) is undecided	assumption
7	Whether email is needed at all alongside WhatsApp is undecided	assumption

Suggested features

#	FEATURE	TYPE
1	Package Balance Tracker	suggested
2	Session / Treatment Notes	suggested
3	Missed / Cancelled Visit Tracking	suggested
4	Payment Mode Tracking	suggested
5	Recurring Visit Scheduling	suggested
6	Simple Patient Search	suggested
7	App Lock (PIN / Fingerprint)	suggested
8	Backup / Export to Excel	suggested

This document covers architecture and feature scope only — no build timeline, no pricing, no visual/UI design yet. Once everything above is confirmed, changed, or dropped, the next step is turning this into an actual development plan.

DZen Doctor App — Architecture Blueprint · Rev A · Draft for review — not yet approved